

Engram: An Integrated Architecture for Continually-Learning LLM Serving via Routed Skill Adapters and Grounded Retrieval

Abstract. Deployed large language models (LLMs) are trained once and frozen. Retrieval-augmented generation (RAG) bolts fresh *facts* onto a frozen model's context but cannot teach it new *skills*; parameter-efficient fine-tuning (PEFT) bakes in skills but goes stale, forgets, and requires an offline retrain. Neither, alone, yields a model that *keeps learning* after deployment. We present **Engram**, an open-source serving architecture that integrates four components behind a single OpenAI-compatible interface: (i) a bring-your-own open-weight base model exposed through a thin driver abstraction; (ii) a routed library of skill LoRA adapters; (iii) grounded retrieval over a vector index and a multi-hop knowledge graph; and (iv) an offline consolidation loop that distills interactions into new skill adapters, gated by evaluation and promoted through a canary lifecycle. Engram's design is constrained by four principles we derive empirically and then validate on a real 4B-parameter model: RAG presupposes an *induction-capable* base; *facts belong in retrieval and skills belong in adapters*; a naively-trained adapter *interferes* with retrieval unless trained context-preservingly; and adapter routing should use *frozen features*, not a trained classifier. We further address a property that is easy to get silently wrong in such systems—skill adapters are bound to the exact base they were trained on—with a fingerprinting guard and a `rebase` operation that retrains skills onto a new base from retained source. A survey of current models indicates that the *integrated expandable-routed-continual-LoRA-memory* system Engram implements is not deployed in any current production or flagship model; Engram is, to our knowledge, the first end-to-end open reference implementation. We report proof-of-concept validation rather than large-scale benchmarks, and discuss the limitations this implies.

1. Introduction

The dominant LLM deployment pattern is *train-once, serve-frozen*. A base model is pretrained and instruction-tuned at great expense, then served read-only. Two mechanisms are used to specialize or update such a model after the fact:

1. **Retrieval-augmented generation (RAG)** [Lewis et al., 2020] places relevant documents into the context window at inference time. RAG is excellent for *knowledge that changes*—it answers questions about facts the model never saw in pretraining—but it does not change the model's *behavior*: it cannot teach a new skill, style, or procedure, and every query pays the token cost of its evidence.
2. **Parameter-efficient fine-tuning (PEFT)**, especially **LoRA** [Hu et al., 2021], trains a small low-rank delta on top of frozen base weights. LoRA is excellent for *skills and behaviors*, but a trained adapter is static: it goes stale, it cannot answer facts outside its training set, and accumulating many adapters raises the unsolved problems of *which adapter to use* and *how to serve many at once*.

A system that *continually learns* in deployment needs both, plus the connective tissue between them: a way to decide what becomes a fact (retrieval) versus a skill (an adapter), a way to route among a growing library of skills, a way to turn raw interactions into new skills offline, and a way to do all of

this safely behind an ordinary chat API. Each individual piece exists in the literature—RAG, LoRA, LoRA routing [Huang et al., 2023; Ostapenko et al., 2024; Muqeeth et al., 2024], multi-LoRA serving [Sheng et al., 2023; Chen et al., 2023], GraphRAG [Edge et al., 2024]—but, as we argue in §2, the *integrated* system is not deployed.

This paper makes three contributions:

- **A set of design principles (§3)** for integrated continual-learning serving, each derived from a controlled experiment and then reproduced on a real model. These principles are not obvious and several contradict naive intuition.
- **An architecture and open-source implementation (§4–§6)**, *Engram*, that realizes these principles: a base-model driver, a capability registry, a deterministic-or-agentic orchestrator, pluggable retrieval (local or vector-DB) and a knowledge graph, an offline consolidation engine with an evaluation gate and canary lifecycle, pluggable persistence (local files or AWS S3 + DynamoDB), multi-tenancy with per-tenant isolation and quotas, and a base-model fingerprint/ rebase safety mechanism.
- **A validation (§7)** on Qwen3.5-4B demonstrating each principle and the end-to-end teach→consolidate→route loop, together with an explicit, honest account (§8) of what was *not* measured.

2. Background and Related Work

Retrieval and induction. RAG [Lewis et al., 2020] retrieves documents and conditions generation on them. Crucially, a model can only *use* retrieved context if it has the in-context copying capability associated with **induction heads** [Olsson et al., 2022]—the circuit that attends to and copies earlier tokens. Pure linear-attention or state-space models can be weak at exact in-context recall [Gu & Dao, 2023], which is why modern efficient models are *hybrids* that interleave a minority of softmax-attention layers among linear-attention or SSM layers [Lieber et al., 2024; De et al., 2024]. Engram treats induction-capability as a *precondition* it checks, not an assumption.

Adapters and adapter routing. LoRA [Hu et al., 2021] is the standard skill module. Composing or routing among many adapters is an active area: LoRAHub [Huang et al., 2023] learns mixing weights per task; the *library of LoRAs* line, including **Arrow** routing [Ostapenko et al., 2024], routes by adapter-intrinsic features; **PHATGOOSE** [Muqeeth et al., 2024] learns per-token gates for zero-shot routing. A recurring difficulty is *routing generalization*: a routing classifier trained on today's adapters does not necessarily route correctly to adapters added tomorrow. Engram adopts *frozen-feature* routing (a capability's routing key is an embedding of its description/source, compared by cosine to the query) precisely to avoid retraining the router as the library grows.

Multi-LoRA serving. S-LoRA [Sheng et al., 2023] and Punica [Chen et al., 2023] serve thousands of adapters concurrently with batched, paged execution; vLLM [Kwon et al., 2023] provides continuous batching and dynamic LoRA loading. Engram's serving driver targets this style of backend for high-throughput nodes, while training remains on a separate node.

Continual learning and consolidation. Naive sequential fine-tuning causes *catastrophic forgetting* [McCloskey & Cohen, 1989; French, 1999]. Neuroscience's **Complementary Learning Systems** theory [McClelland et al., 1995; Kumaran et al., 2016]—a fast hippocampal store that is gradually *consolidated* into slow neocortex—motivates Engram's split between an immediately-updatable retrieval store ("fast") and slowly-consolidated skill adapters ("slow"). HippoRAG

[Gutiérrez et al., 2024] applies a related neurobiological framing to retrieval; Engram applies it to the *retrieval-vs-adaptor* division and to the offline consolidation schedule.

Graph-structured retrieval. GraphRAG [Edge et al., 2024] extracts an entity–relation graph from a corpus and answers queries by traversing it, enabling multi-hop reasoning that flat vector retrieval misses. Engram incorporates a lightweight triple store with multi-hop query and exposes it both as retrieval context and as an agent tool.

Mixture-of-Experts and model "memory." Production MoE models [Shazeer et al., 2017; Jiang et al., 2024] route tokens among *fixed* experts trained jointly; they are not an *expandable* library updated post-deployment. Product "memory" features in deployed assistants are text notes injected into context—i.e., a form of RAG—not parametric skills. We therefore distinguish Engram's *expandable, routed, parametric, continually-consolidated* memory from both.

The gap. Surveying the above, every component is individually mature, but we find no deployed model that *integrates* (a) a library of skill adapters that *grows continually*, (b) routed by a mechanism that *generalizes to adapters added after the router was built*, (c) consolidated from interactions as a *continual-learning long-term memory*, (d) alongside text/graph retrieval for facts, (e) served behind a normal API. Engram is a reference implementation of that integration.

3. Design Principles

We state four principles, each as a falsifiable claim, with the experiment that motivates it. §7 reports the measurements.

P1 — RAG = retrieval + induction. Retrieval is useless unless the base can copy from context. A serving system must *verify* induction-capability of a bring-your-own base rather than assume it. *Engram check:* the driver exposes `arch_info().induction_capable`, derived from the presence of softmax-attention layers; the platform warns if a base lacks it.

P2 — Facts to retrieval, skills to adapters. New *facts* should enter retrieval (instantly available, generalizes to unseen facts); new *skills/behaviors* should enter adapters (consolidated offline). Putting facts in adapters fails to answer facts the adapter never trained on; putting skills only in context wastes tokens and does not change behavior.

P3 — Adapters and retrieval interfere; train context-preservingly. A LoRA trained only to *memorize* facts can teach the model to ignore retrieved context, collapsing RAG performance. Training the adapter on a *mixture* of memorization and generic "answer-from-the-provided-context" examples preserves the model's retrieval-use behavior. Engram's consolidation always injects such context-preserving examples.

P4 — Route with frozen features, not a trained classifier. A routing classifier trained on the current adapter set tends to overfit and misroute to adapters added later. A routing key derived from a *frozen* embedding of the capability (its description/source) compared to the query embedding generalizes to newly-added adapters without retraining the router.

A fifth principle emerges from operating such a system and is treated separately in §5.4:

P5 — Skills are base-bound; bind them explicitly. A LoRA delta is meaningful only relative to the exact base weights it was trained against. Changing the base silently invalidates every adapter. A serving system must *fingerprint* the base, refuse to serve mismatched adapters, and retain enough to *retrain* them.

4. System Architecture

Engram is organized around a small number of interfaces, each with swappable implementations. The request path is: *client* → *API* (*auth*, *tenancy*, *quotas*) → *orchestrator* → {*registry*, *retrieval*, *graph*, *driver*} → *grounded answer*. The learning path is: *teach/ingest* → *retrieval* (+ *graph*) *immediately*, and a *queued job* → *consolidation* → *eval-gate* → *canary* → *live*.

4.1 Base-LLM Driver. Every backend implements one interface: `generate`, `load_lora`, `activate_lora`, `train_lora`, `embed`, `capabilities()`, `arch_info()`, and `fingerprint()`. The orchestrator talks only to this interface, which is what makes base models plug-and-play. A `PEFTDriver` (transformers + PEFT) both trains and serves; a `VLLMDriver` serves at high throughput with hot-swappable adapters but does not train. `capabilities()` lets the orchestrator degrade gracefully (e.g., an inference-only node refuses `train_lora`). `arch_info()` reports induction-capability (P1).

4.2 Capability Registry. Everything added to Engram—a skill (LoRA), a knowledge source (an index), or a tool (a callable)—is a *registered capability* with metadata, a frozen-feature routing key (P4), a lifecycle status, and, for skills, a base fingerprint and retained source (P5). Adding or removing a capability requires no code change. In agentic mode, live capabilities are surfaced to the model as tools.

4.3 Orchestrator. The default (deterministic) orchestrator runs *gate* → *retrieve* → *route* → *assemble* → *generate*. Retrieval is **relevance-gated**: if nothing scores above a threshold, *nothing* is injected, since irrelevant context measurably degrades answers. Routing scores the query embedding against each live skill's frozen routing key (P4). The assembled prompt contains retrieved passages and multi-hop graph facts; the response carries *provenance* (which documents, which skill, which graph facts). An **agentic** orchestrator instead exposes tools (a built-in `search_knowledge`, a `graph_search`, and any registered tools) and lets the model drive a tool-calling loop; it supports token streaming with marker-safe holdback so tool-call syntax never leaks into the user stream.

4.4 Retrieval and Knowledge Graph. Retrieval embeds text and performs gated nearest-neighbor search. The vector store is *interchangeable* behind a single `add/retrieve/count` interface—local FAISS by default, or a networked vector DB (Qdrant; AWS OpenSearch) selected by configuration. The knowledge graph stores (subject, relation, object) triples extracted by the base model itself; a query matches entities mentioned in it and walks *k* hops to gather connected facts, surfacing multi-hop relationships that flat retrieval misses.

4.5 Consolidation, Eval-gate, Lifecycle. Consolidation turns a teach/ingest job into a skill: it self-distills question/answer pairs from the source, adds context-preserving examples (P3), trains a LoRA, and submits it to an **evaluation gate**. Only on passing is the skill registered, stamped with the base fingerprint and source (P5). Promotion follows a **canary lifecycle**: a new skill serves a configurable fraction of matching traffic (`canary`) before being promoted to `live` or rolled back. Consolidation runs under a policy—`manual`, `threshold` (after *N* queued), or `scheduled` (every interval)—on a background worker that shares a model-lock with serving so training never races generation on one GPU.

4.6 Persistence, Multi-tenancy, Connectors. State persists behind a `Store` interface: local files by default, or AWS S3 (blobs: indexes, adapter weights) + DynamoDB (registry) when configured—nodes are stateless and reload shared state, enabling horizontal scale. Multi-tenancy issues per-tenant API keys (stored only as hashes); a key maps to a tenant's *project namespace*, so isolation is enforced from the key rather than from any request field, and per-tenant rate and resource quotas are

applied server-side. Connectors ingest real documents (files, directories, PDF, HTML, code) into retrieval and, optionally, the graph.

5. The Continual-Learning Loop

5.1 Teach-in-band. A user may, mid-conversation through the chat API, instruct the system to internalize content. Retrieval ingestion is *immediate*—the fact is answerable on the next turn. Skill internalization is *queued*: training a LoRA cannot complete within a chat turn, so creation is triggered in-band while consolidation completes asynchronously, with retrieval covering the interval. This is the operational realization of P2: facts are instant (retrieval); skills bake (adapters).

5.2 Division of labor. The same content can contribute to both stores: its facts to retrieval (instantly, generalizing to unseen queries) and its skill/behavior to an adapter (after consolidation). The orchestrator can then answer from retrieval, from a routed skill, or both, with provenance attributing each.

5.3 Consolidation as CLS. The retrieval store is the fast, immediately-writable memory; skill adapters are the slow, consolidated memory; the consolidation policy is the scheduled transfer between them—a direct analogue of Complementary Learning Systems.

5.4 Base changes and rebase (P5). Because adapters are base-bound, the driver computes a `fingerprint` (model id plus architecture dimensions) and every skill records the fingerprint it was trained under and its source text. On load, a skill whose fingerprint differs from the current base is marked **incompatible** and *not served*—replacing silent corruption with explicit failure. `rebase()` then retrains each incompatible skill from its retained source onto the new base; retrieval and graph state, being base-agnostic, are untouched. Thus a base swap costs training time, not knowledge.

6. Implementation

Engram is implemented in Python with a small dependency core and optional extras per backend (PEFT, vLLM, RAG, vector DBs, AWS, document parsers, serving). It exposes an OpenAI-compatible server (`/v1/chat/completions` with streaming, plus `teach`, `ingest`, `consolidate`, `promote`, `rollback`, `rebase`, and `capabilities`), so any existing client can drive it unchanged. The reference configuration runs a single workstation; the same code scales to a trainer node plus stateless vLLM serving nodes over a shared AWS store. The implementation is open source under Apache-2.0 with continuous integration and a unit-test suite covering the registry, persistence (including AWS via mocked S3/DynamoDB), orchestration and routing, the agent loop and its tool-call parsing, the knowledge graph, multi-tenancy and quotas, the vector-backend factory (with a real in-memory Qdrant round-trip), and the base-fingerprint/rebase guards.

7. Validation

We validate the design principles and the end-to-end loop on **Qwen3.5-4B**, a hybrid model (24 linear-attention + 8 softmax-attention language layers), on a single RTX 5090. These are *proof-of-concept* experiments using small synthetic fact sets, intended to demonstrate the qualitative phenomena the design depends on, not to establish benchmark rankings.

7.1 Facts vs. skills (P2), and interference (P3). We construct fabricated facts of the form *"the capital of ⟨nonce⟩ is ⟨nonce⟩"* so that the base provably cannot know them, split into *seen* (used to

train a LoRA) and *fresh* (held out; available only via retrieval). We measure exact-answer accuracy under four configurations:

Configuration	Seen facts	Fresh facts
Base (no augmentation)	0%	0%
Base + RAG	90%	88%
Base + LoRA	58%	0%
Base + LoRA + RAG (context-preserving)	82%	72%

The base scores zero on both, confirming the facts are genuinely novel. **RAG answers both seen and fresh** (90%/88%): retrieval *generalizes* to facts never trained on (P2). **A LoRA answers only seen facts** (58%/0%): fine-tuning cannot recall what it never trained on (P2). A *memorization-only* LoRA combined with RAG exhibited **interference**—fresh-fact accuracy collapsed (to 0% in an earlier run)—because the adapter taught the model to ignore retrieved context; training the adapter with **context-preserving** examples (P3) recovered fresh accuracy to 72% while retaining 82% on seen. This both reproduces the failure and validates the fix.

7.2 The teach→consolidate→route loop. Taught a fact in-band, the system (i) ingested it to retrieval immediately and (ii) queued consolidation. Consolidation self-distilled six Q/A pairs, added context-preserving examples, trained a LoRA, and passed the evaluation gate at 1.0, registering a routable skill. A subsequent query was *routed to the new skill via its frozen-feature key* (P4) and answered correctly with provenance attributing both the retrieved document and the routed skill. This demonstrates the full cycle—in-band teaching, offline consolidation, eval-gated registration, frozen-feature routing—on a real model.

7.3 Multi-hop GraphRAG. From two separately-taught statements (*A manages B*; *B owns C*), the base extracted two clean triples; a query naming *C* induced a two-hop graph walk $C \rightarrow B \rightarrow A$ that surfaced both connecting facts, and the model answered the composed question correctly. The agentic `graph_search` tool reproduced the result via an explicit tool call.

7.4 Base-binding guard (P5). Loading a registry of skills under a *different* base fingerprint marked those skills `incompatible` and excluded them from serving; loading under the *same* fingerprint kept them live; `rebase()` re-queued only the incompatible skills that retained source. This confirms the guard converts a silent failure into an explicit, recoverable one.

8. Discussion and Limitations

We are deliberately explicit about scope.

- **Scale of evaluation.** §7 uses small synthetic fact sets and a single 4B model. The experiments demonstrate the *phenomena* the architecture relies on (and their fixes), but do not constitute large-scale benchmarks; absolute percentages should be read as illustrative. Establishing performance on standard knowledge-intensive and multi-task suites, at larger model scale and library size, is future work.
- **Routing at scale.** Frozen-feature routing (P4) avoids router retraining, but its accuracy as the library grows to hundreds or thousands of skills—the central open problem in the LoRA-library literature—is unmeasured here.

- **Serving backend.** The high-throughput `VLLMDriver` is implemented to vLLM's documented API but is *untested in this work* (the development platform could not run vLLM); likewise the AWS OpenSearch vector backend is written but not exercised against a live endpoint. The FAISS and Qdrant backends, the AWS persistence path (against mocked services), and the single-node PEFT path are validated.
- **Fingerprint granularity (P5).** The base fingerprint (model id + architecture dimensions) detects different-model and different-size swaps but not a same-identifier, silently-different-weights swap; a content hash of selected base tensors would close this minor gap.
- **Single-node consolidation.** A background consolidation shares a model-lock with serving, so on one GPU it pauses generation; the intended production topology separates a trainer node from serving nodes.
- **Compliance.** Regulated-deployment requirements (e.g., HIPAA/GDPR/FedRAMP) are *designed for* (data locality, tenant isolation, provenance) but not certified.

None of these undercut the paper's claim, which is about *integration and design*: that the four components can be combined into a coherent, continually-learning serving system governed by a small set of empirically-grounded principles, and that doing so is both possible and, per our survey, not yet done in deployed models.

9. Conclusion

Engram demonstrates that a frozen base model can be turned into a continually-learning system by combining grounded retrieval (for facts), a routed library of skill adapters (for behaviors), a knowledge graph (for multi-hop structure), and an offline consolidation loop (for turning interactions into skills), behind an ordinary chat API—provided one respects a few non-obvious principles: verify induction-capability, separate facts from skills, train adapters context-preservingly so they do not suppress retrieval, route by frozen features so the router need not be retrained as the library grows, and fingerprint adapters to their base so a swap fails loudly and recovers. We provide the design, an open-source implementation with these principles enforced in code, and proof-of-concept validation on a real model. The integrated system this realizes is, to our knowledge, absent from current deployed models; we offer Engram as a reference point and a substrate for the larger-scale study that the open problems—routing at scale, multi-adapter interference, consolidation schedules—now require.

Availability

Source, tests, and documentation: <https://github.com/RajaNarsimham/engram> (Apache-2.0).

References

1. P. Lewis et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS, 2020.
2. E. J. Hu et al. *LoRA: Low-Rank Adaptation of Large Language Models*. ICLR, 2022.
3. C. Olsson et al. *In-context Learning and Induction Heads*. Transformer Circuits / Anthropic, 2022.

4. C. Huang et al. *LoRAHub: Efficient Cross-Task Generalization via Dynamic LoRA Composition*. 2023.
5. O. Ostapenko et al. *Towards Modular LLMs by Building and Reusing a Library of LoRAs*. ICML, 2024.
6. M. Muqeeth et al. *Learning to Route Among Specialized Experts for Zero-Shot Generalization (PHATGOOSE)*. ICML, 2024.
7. Y. Sheng et al. *S-LoRA: Serving Thousands of Concurrent LoRA Adapters*. MLSys, 2024.
8. L. Chen et al. *Punica: Multi-Tenant LoRA Serving*. MLSys, 2024.
9. W. Kwon et al. *Efficient Memory Management for Large Language Model Serving with PagedAttention (vLLM)*. SOSP, 2023.
10. D. Edge et al. *From Local to Global: A Graph RAG Approach to Query-Focused Summarization*. 2024.
11. J. McClelland, B. McNaughton, R. O'Reilly. *Why there are complementary learning systems in the hippocampus and neocortex*. Psychological Review, 1995.
12. D. Kumaran, D. Hassabis, J. McClelland. *What Learning Systems do Intelligent Agents Need? Complementary Learning Systems Theory Updated*. Trends in Cognitive Sciences, 2016.
13. M. McCloskey, N. Cohen. *Catastrophic Interference in Connectionist Networks*. Psychology of Learning and Motivation, 1989.
14. R. French. *Catastrophic Forgetting in Connectionist Networks*. Trends in Cognitive Sciences, 1999.
15. B. Gutiérrez et al. *HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models*. NeurIPS, 2024.
16. A. Gu, T. Dao. *Mamba: Linear-Time Sequence Modeling with Selective State Spaces*. 2023.
17. O. Lieber et al. *Jamba: A Hybrid Transformer-Mamba Language Model*. 2024.
18. S. De et al. *Griffin: Mixing Gated Linear Recurrences with Local Attention for Efficient Language Models*. 2024.
19. N. Shazeer et al. *Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer*. ICLR, 2017.
20. A. Jiang et al. *Mixtral of Experts*. 2024.

This paper documents the design and proof-of-concept validation of an open-source system. Empirical results are at proof-of-concept scale (§8); they demonstrate the qualitative phenomena the architecture depends on and are not large-scale benchmarks.